Programming for AI	Lab 05 **File Handling, TXT/CSV/JSON Files, Web APIs, API Requests & Responses, JSON Parsing, Exception Handling, and Error Handling.**

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

FALL 2026AIMS AND OBJECTIVES

- To understand how to read data from and write data to local files using Python.
- To learn basic file handling for TXT, CSV, and JSON files.
- To understand the fundamentals of interacting with web APIs using Python.
- To learn how to fetch live data from an API, handle responses, and process JSON data

INTRODUCTION

In this lab we will be covering the following topics
Python provides simple and powerful tools for working with both local files and online data sources. In this lab, students will learn how to read data from and write data to TXT, CSV, and JSON files. File handling is an important skill because many applications need to store and retrieve data from external files. Students will also be introduced to **web APIs (Application Programming Interfaces)**, which allow Python programs to communicate with online services and retrieve live data. Using Python's HTTP request capabilities, students will fetch data from an API, handle the response, parse JSON data into Python objects such as dictionaries and lists, and process the retrieved information. These concepts form the foundation for developing data-driven Python applications.

FUNCTIONS

Built-in String Functions

- capitalize() - Converts the first character to upper case.
- casefold() - Converts string into lower case.
- center() - Returns a centered string.
- count() - Returns the number of times a specified value occurs in a string.
- endswith() - Returns true if the string ends with the specified value.
- find() - Searches the string for a specified value and returns the position of where it was found.
- format() - Formats specified values in a string.
- format_map() – This method is used to format specified values in a string by using a mapping (usually a dictionary) to replace placeholders with corresponding values. It is similar to the format() method, but instead of positional or keyword arguments, it takes a mapping object as its argument.
- index() - Searches the string for a specified value and returns the position of where it was found.
- isalnum() - Returns True if all characters in the string are alphanumeric.
- isalpha() - Returns True if all characters in the string are in the alphabet.
- isdigit() - Returns True if all characters in the string are digits.
- islower() - Returns True if all characters in the string are lower case.
- isupper() - Returns True if all characters in the string are upper case.
- isnumeric() - Returns True if all characters in the string are numeric.
- isspace() - Returns True if all characters in the string are whitespaces.

- istitle() - Returns True if the string follows the rules of a title.
- join() - Joins the elements of an iterable to the end of the string.
- lower() - Converts a string into lower case.
- upper() - Converts a string into upper case.
- replace() - Returns a string where a specified value is replaced with a specified value.
- rfind()- Searches the string for a specified value and returns the last position of where it was found.
- 23)split() - Splits the string at the specified separator, and returns a list.
- 24)splitlines() - Splits the string at line breaks and returns a list.
- 25)startswith() - Returns true if the string starts with the specified value.
- 26) title() - Converts the first character of each word to upper case.
- 27) zfill() Fills the string with a specified number of 0 values at the beginning.

EXCEPTION HANDLING

Python has many built-in exceptions that are raised when your program encounters an error (Something in the program goes wrong). When these exceptions occur, the Python interpreter stops the current process and passes it to the Calling process until it is handled. If not handled, the program will crash.

```
try:
    number = int(input("Enter a number: "))
    result = 100 / number
    print(result)
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("Number cannot be zero.")
```

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid input.")
else:
    print("Valid number entered.")
```

```
try:
    number = 10 / 2
    print(number)
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Program finished.")
```

FILING IN PYTHON(TXT , JSON , CSV)

A Python program normally stores its data temporarily in variables. When the program terminates, the data stored in variables is lost. If we want to store data permanently so that it can be used later, we can save it in a **file**.**File handling** refers to the process of creating, opening, reading, writing, modifying, and closing files using a programming language.Python provides a built-in open() function for working with files. Files can contain different types of information, such as text, numbers, records, or structured data.Common file formats used in Python applications include:

1. **TXT** — simple text data
2. **CSV** — tabular data arranged in rows and columns
3. **JSON** — structured data commonly used for data exchange and APIs

Mode	Description
r	Opens a file for reading
w	Opens a file for writing
a	Opens a file for appending
x	Creates a new file
r+	Opens a file for reading and writing
w+	Opens a file for writing and reading
a+	Opens a file for appending and reading
b	Binary mode
t	Text mode

CREATE A FILE

```
# Create a new TXT file
file = open("students.txt", "w")
# Close the file
file.close()
print("File created successfully.")
```

CHECK IF FILE IS EXISTING OR NOT

```
import os
if os.path.exists("students.txt"):
    print("File exists.")
else:
    print("File does not exist.")
file_path = r"C:\Users\Student\Documents\PythonLab\students.txt"
if os.path.exists(file_path):
    print("File exists.")
else:
    print("File does not exist.")
```

Note: os.path.exists returns True for both files and directories

Os.path.isfile() os.path.isdir()

WRITING ON FILE

```
file_path = "students.txt"
if not os.path.exists(file_path):
    with open(file_path, "w") as file:
        file.write("Ali, 20\n")
        file.write("Sara, 21\n")
        file.write("Ahmed, 22\n")

    print("File created successfully.")
else:
    print("File already exists.")
```

READING A FILE

Function / Method	working
open()	Opens a file in a specified mode such as read, write, or append.
read()	Reads the entire file content and returns it as a string.
read(size)	Reads a specified number of characters from the file.
readline()	Reads one line from the file at a time.
readlines()	Reads all lines from the file and returns them as a list.
for line in file	Reads and processes a file one line at a time.
seek()	Moves the file cursor to a specified position.
tell()	Returns the current position of the file cursor.
close()	Closes the opened file and releases system resources.
strip()	Removes leading and trailing whitespace, including \n, from a string.
splitlines()	Splits a string into a list of lines without keeping the newline characters.
enumerate()	Adds a counter/index to each line while iterating through a file.
write()	Writes a string to a file.
writelines()	Writes multiple strings to a file without automatically adding newlines.
os.remove()	Deletes a specified file from the system.

```
print("\nStudent Records:")

with open(file_path, "r") as file:
    data = file.read()
    print(data)
```

UPDATING A FILE

```
with open(file_path, "r") as file:
    data = file.read()
data = data.replace("Ali, 20", "Ali, 21")
with open(file_path, "w") as file:
    file.write(data)
print("Student record updated successfully.")
```

DELETE EXISTING FILE

```
if os.path.exists(file_path):
    os.remove(file_path)
    print("File deleted successfully.")
else:
    print("File does not exist.")
```

JSON FILLING

Python has a built-in package called JSON, which can be used to work with JSON data. JavaScript Object Notation (JSON) is a standardized format commonly used to transfer data as text that can be sent over a network. It's used by lots of APIs and Databases, and it's easy for both humans and machines to read.

```
with open("student.json", "w") as file:
    pass
print("JSON file created successfully.")
```

WRITING ON JSON FILE

```
student = {  "name": "Ali", "age": 20, "city": "Karachi"}
with open("student.json", "w") as file:
    json.dump(student, file, indent=4)
print("Student data saved successfully.")
```

READ A JSON FILE

```
with open("student.json", "r") as file:
    student = json.load(file)
print(student)
```

HOW TO ADD MULTIPLE RECORDS TO JSON

```
students = [{"id": 1,"name": "Ali","age": 20},
                        {"id": 2,"name": "Sara","age": 21},
                        { "id": 3,"name": "Ahmed","age": 22 }]
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
print("Students created successfully.")
```

READ RECORDS INDIVIDUALLY

```
with open("students.json", "r") as file:
    students = json.load(file)

for student in students:
    print("ID:", student["id"])
    print("Name:", student["name"])
    print("Age:", student["age"])
    print("----------------")
```

UPDATE EXISTING JSON

```
with open("students.json", "r") as file:
    students = json.load(file)

for student in students:
    if student["id"] == 1:
        student["age"] = 21

with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

print("Student updated successfully.")
```

DELETING DATA FROM JSON

```
with open("students.json", "r") as file:
    students = json.load(file)
students = [
    student for student in students
    if student["id"] != 2
]
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
print("Student deleted successfully.")
```

Function / Method	
json.dump()	Converts a Python object into JSON format and writes it directly to a file.
json.dumps()	Converts a Python object into a JSON-formatted string.
json.load()	Reads JSON data from a file and converts it into a Python object.
json.loads()	Converts a JSON-formatted string into a Python object.
Option	One-line Definition
indent	Adds indentation to JSON output to make it easier to read.
sort_keys	Sorts dictionary keys alphabetically when writing JSON.
ensure_ascii	Controls whether non-ASCII characters are converted into Unicode escape sequences.
encoding="utf-8"	Specifies UTF-8 encoding so the file can correctly handle Unicode characters.

CSV FILES

CSV (Comma-Separated Values) is a plain-text file format used to store and exchange tabular data in rows and columns, where individual values are typically separated by commas.

```
import csv
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
print("CSV file created successfully.")

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["ID", "Name", "Age", "Department"])
    writer.writerow([1, "Ali", 20, "Computer Science"])
    writer.writerow([2, "Sara", 21, "Software Engineering"])
    writer.writerow([3, "Ahmed", 22, "Data Science"])
print("CSV file created successfully.")
```

```
with open("students.csv", "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

CSV Function / Method

csv.reader()	Reads CSV data row by row and returns each row as a list.
csv.writer()	Creates a CSV writer object for writing list-based rows into a CSV file.

writer.writerow()	Writes a single row of data into a CSV file.
writer.writerows()	Writes multiple rows of data into a CSV file.
csv.DictReader()	Reads CSV rows as dictionaries using the first row as column names.
csv.DictWriter()	Writes dictionary data into a CSV file according to specified column names.
writer.writeheader()	Writes the specified field names as the header row of a CSV file.
next()	Retrieves the next row from a CSV reader, commonly used to skip the header.

WEB API

A Web API (Application Programming Interface) is a set of rules and endpoints that allows a Python program to communicate with a web server, request data, and receive data over the internet**.**

API ENDPOINT

An **API endpoint** is a URL where a particular API resource can be accessed.

HTTP METHODS

HTTP methods specify what operation we want to perform.

Method	Purpose	Example
GET	Retrieve data	Get users
POST	Create new data	Add a user
PUT	Replace existing data	Replace a user
PATCH	Update part of data	Change user's email
DELETE	Delete data	Delete a user

INSTALLING REQUESTS

pip install requests

```
import requests
url = "https://jsonplaceholder.typicode.com/users"
response = requests.get(url)
print(response)
```

Status Code	Meaning
200	Request successful
201	Resource successfully created
204	Successful request with no content
400	Bad request
401	Unauthorized
403	Forbidden
404	Resource not found
500	Internal server error

```
url = "https://jsonplaceholder.typicode.com/users"
user = {
    "name": "Ali",
    "email": "ali@gmail.com"
}
```

```
response = requests.post(url, json=user)
print(response.status_code)
print(response.json())
```

```
url = "https://jsonplaceholder.typicode.com/users/1"
user = {
    "name": "Ahmed",
    "email": "ahmed@gmail.com"
}
response = requests.put(url, json=user)
print(response.status_code)
print(response.json())
```

```
url = "https://jsonplaceholder.typicode.com/users/1"

response = requests.delete(url)

print(response.status_code)
```

```
try:
    response = requests.get(
        "https://jsonplaceholder.typicode.com/users",
        timeout=5
    )
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.Timeout:
    print("Request timed out.")
except requests.exceptions.ConnectionError:
    print("Connection error.")

except requests.exceptions.HTTPError:
    print("HTTP error occurred.")
except requests.exceptions.RequestException as error:
    print("Request failed:", error)d
```

Lab Tasks:

1. During an AI experiment, researchers need to maintain configuration information for different machine learning experiments in a text file named experiments.txt. Each experiment contains an experiment ID, model name, dataset name, learning rate, and number of epochs. Develop a Python program that creates the file, stores multiple experiment configurations, reads and displays the configurations, searches for an experiment by its ID, adds a new experiment, and updates an existing experiment's parameters. The program should also allow a selected experiment to be removed from the file while handling situations where the requested experiment does not exist.

2. A machine learning researcher has collected a CSV dataset containing information about different patients, including age, BMI, blood pressure, glucose level, and a diabetes prediction label. Develop a Python program that reads the dataset using appropriate CSV methods, displays the records, calculates the number of positive and negative cases, identifies patients whose glucose level is above a specified threshold, and calculates the average BMI and glucose level. The program should also allow a new data record to be added and an existing record to be updated based on its ID. Use csv.reader() or csv.DictReader() appropriately and handle the CSV data as Python lists or dictionaries.

3. After training several machine learning models, a researcher stores their evaluation results in a JSON file named model_results.json. Each record contains the model name, accuracy, precision, recall, and F1-score. Develop a Python program that reads the JSON file and identifies the model with the highest accuracy and the model with the highest F1-score. The program should also allow a new model result to be added, update the evaluation metrics of an existing model, and remove a model's result when required. Finally, save the modified results back to the JSON file using appropriate JSON functions.

4. An AI application needs to obtain prediction results from an online machine learning API. Develop a Python program that sends a GET request to a suitable public API and processes its JSON response. The program should check the HTTP status code, convert the JSON response into Python objects, and extract specific information from the response, such as the prediction result, confidence score, and input information. Use appropriate query parameters where required and implement exception handling for connection errors, timeouts, HTTP errors, and invalid responses. Display a meaningful message whenever the API cannot provide the requested result.

5. An NLP researcher wants to collect text data from an online API for sentiment-analysis experiments. Develop a Python program that sends requests to a suitable public API to retrieve text-based data, parses the JSON response, and extracts relevant fields such as text, author/source, and other available metadata. The program should allow the user to specify a query or parameter, display the retrieved results, and save the selected text data into both a JSON file and a CSV file for later machine learning experiments. Appropriate exception handling should be implemented for failed requests, timeouts, invalid responses, and file-related errors.

6. A computer vision application uses a REST API to process images and return prediction results. Develop a Python program that demonstrates communication with a suitable API by performing different HTTP operations where supported. The program should retrieve prediction information using GET, send input data using POST, and demonstrate how an existing prediction record could be modified using PUT or PATCH and removed using DELETE. The program should display the HTTP status code and JSON response for each operation and handle possible API and network exceptions using appropriate exception-handling techniques.

7. A researcher is developing an AI experiment pipeline in which data is obtained from an online API, processed in Python, and stored locally for future analysis. Develop a Python program that retrieves suitable AI-related data from a public API using a GET request and query parameters, checks the HTTP response, and parses the returned JSON into Python objects. Extract the required fields and perform a simple analysis, such as calculating an average, counting categories, or identifying the highest-scoring result. Save the processed data into both JSON and CSV files, then read the saved files and verify the stored results. The program must use appropriate exception handling for API connection errors, timeouts, HTTP errors, missing files, and invalid JSON data, and should use try, multiple except blocks, else, and finally appropriately.